

面向大规模查询的数据结构算法

2025 沪疆教育信息化 算法小论文

上海市育才中学 丁梓洋

<https://github.com/NailFec/OIpaper>

2025-06-02

Outline

1. 例题	3	4. 树状数组 / 线段树	20
1.1 题目大意	4	4.1 实现	21
1.2 问题分析	5	4.2 时间复杂度	22
2. 分块	6	5. 归并树	23
2.1 实现	7	5.1 实现	24
2.2 时间复杂度	8	5.2 时间复杂度	25
2.3 衍生题目	9	5.3 优化	26
3. 普通莫队算法	11	6. 扫描线	27
3.1 实现	12	6.1 实现	28
3.2 时间复杂度	13	6.2 时间复杂度	29
3.3 优化	15	7. 总结	30
3.3.1 最优块的大小	15	7.1 总结	31
3.3.2 奇偶块交错排序	16	7.2 参考文献	34
3.3.3 数据离散化	17	7.3 致谢	35
3.3.4 优化 <i>cnt</i> 数组	18		
3.4 衍生题目	19		

Outline

1. 例题
2. 分块
3. 普通莫队算法
4. 树状数组 / 线段树
5. 归并树
6. 扫描线
7. 总结

1.1 题目大意

例题

给定一个长为 N ($1 \leq N \leq 10^6$) 的序列 $\{a_i\}$ ($1 \leq a_i \leq 10^6$), 给定指令: $Q\ l\ r$, 询问区间 $[l, r]$ 内有多少个不同的数。

现有 M ($1 \leq M \leq 10^6$) 条指令。要求对每条询问指令输出其答案。内存限制 512 MB, 时间限制 2.0 s。¹

¹来源: 洛谷 P1972 [SDOI2009] HH 的项链, 有修改

1.2 问题分析

该问题属于典型的区间查询问题，要求高效地处理大量查询，序列长度和询问次数都在 10^6 数量级。一种暴力的思路是对每个询问单独统计不同元素的个数，但这在最坏情况下时间复杂度会达到 $O(NM)$ ，显然无法满足时间限制。

由于题目明确是静态查询，也就是说输入的序列不会发生变化，我们可以设计一个离线算法来处理所有询问。

Outline

1. 例题
2. 分块
3. 普通莫队算法
4. 树状数组 / 线段树
5. 归并树
6. 扫描线
7. 总结

2.1 实现

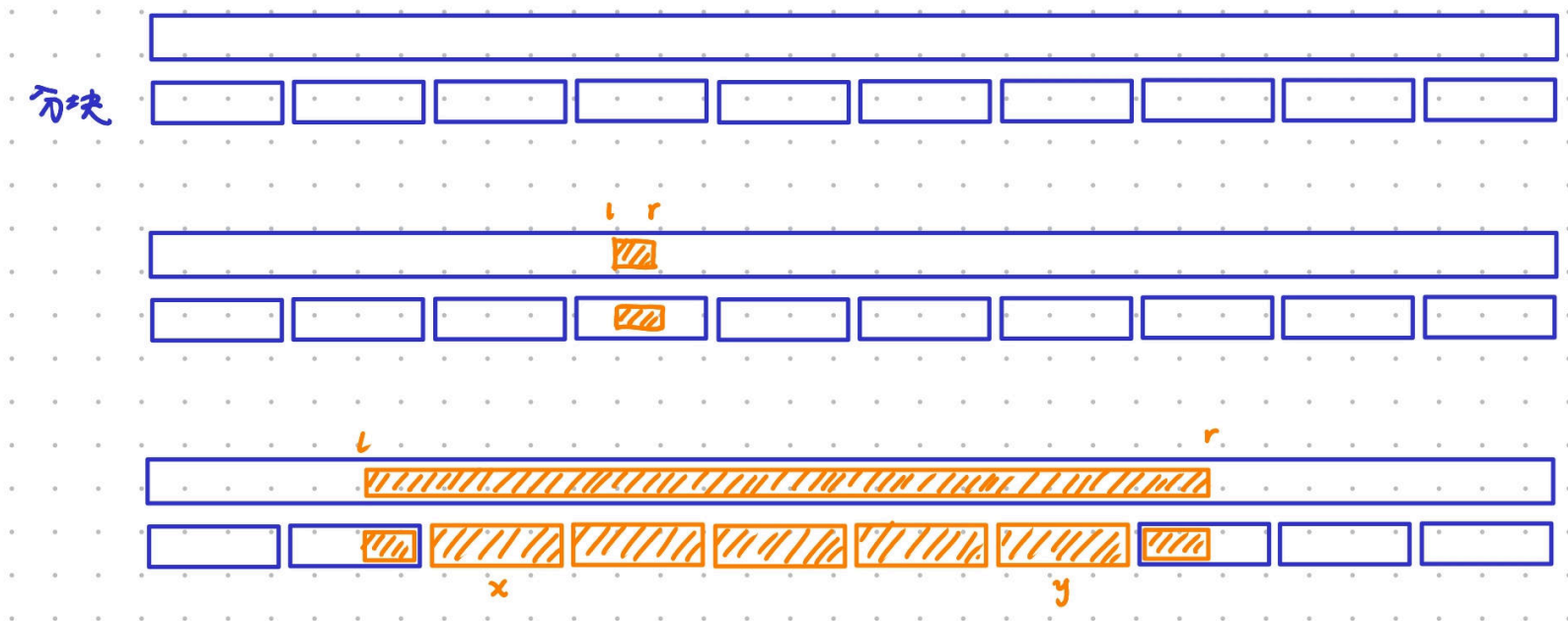


Figure 1: 分块解法简要示意图

2.2 时间复杂度

该算法的时间复杂度为 $O((N + M)\sqrt{N}) = O(N\sqrt{N})$ ，其中 N 为序列长度， M 为询问区间的数量。预处理 bef 和 nxt 数组的时间复杂度为 $O(N)$ ；预处理 f 数组的时间复杂度为 $O(DS^2)$ ，即 $O(N\sqrt{N})$ ；对于所有查询，时间复杂度为 $O(MS)$ ，即 $O(M\sqrt{N})$ 。

2.3 衍生题目

分块是一种灵活且通用的思想。即思考过程相对简单，代码实现逻辑单一直白。尤其在处理一些复杂的查询问题时，能充分利用其灵活性，处理那些普通树状数组或线段树难以做到的情况。对于区间查询和更新问题，分块可以在不需要过度复杂结构的情况下，提供较为高效的解决方案。

分块的一个关键优势在于它能够将问题的复杂度从一个大规模的结构中分解到若干小块中，这样能够更好地适应实际问题中的各种复杂约束。另外，分块不仅限于区间问题，还可以扩展到其他类型的问题，比如求最小公倍数、最大公约数、处理频率分布等。

分块的实现思路和其他数据结构不同，时间复杂度也不同。在应对不同题目时，读者可以根据自己擅长的解法、评估时间复杂度等因素，选择合适的方法。

2.3 衍生题目

以下推荐几道不同难度的题目，均可同时使用分块和数据结构完成，可供读者练习和比较两者的思路：

- 洛谷 P4145 上帝造题的七分钟 2 / 花神游历各国
- 洛谷 P1471 方差
- 洛谷 P1975 [国家集训队] 排队

对于难度更高的大分块题目，推荐：洛谷题单 - YNOI 大分块系列¹。

¹洛谷题单 - YNOI 大分块系列：<https://www.luogu.com.cn/training/44148>

Outline

1. 例题
2. 分块
3. 普通莫队算法
4. 树状数组 / 线段树
5. 归并树
6. 扫描线
7. 总结

3.1 实现

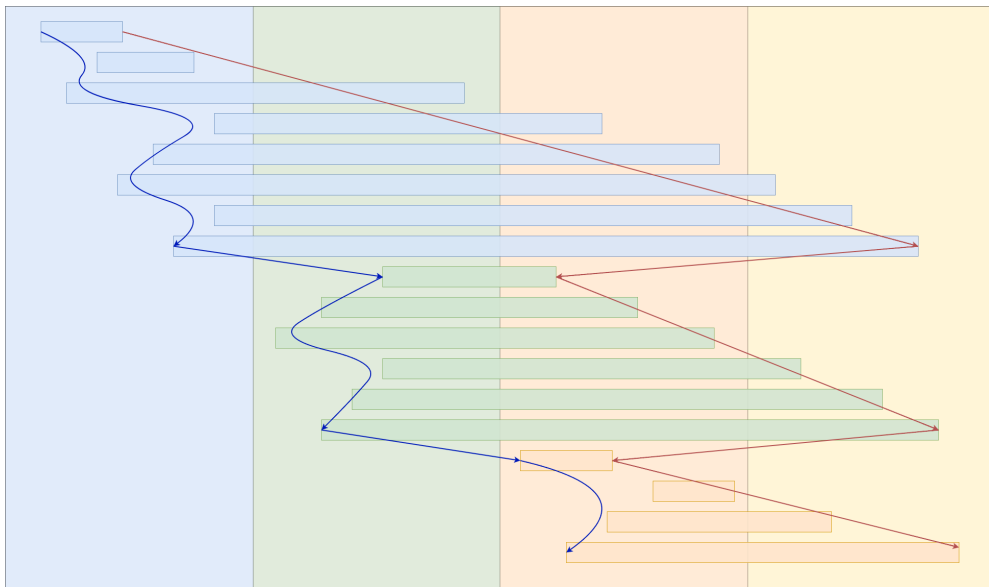


Figure 2: 普通莫队算法

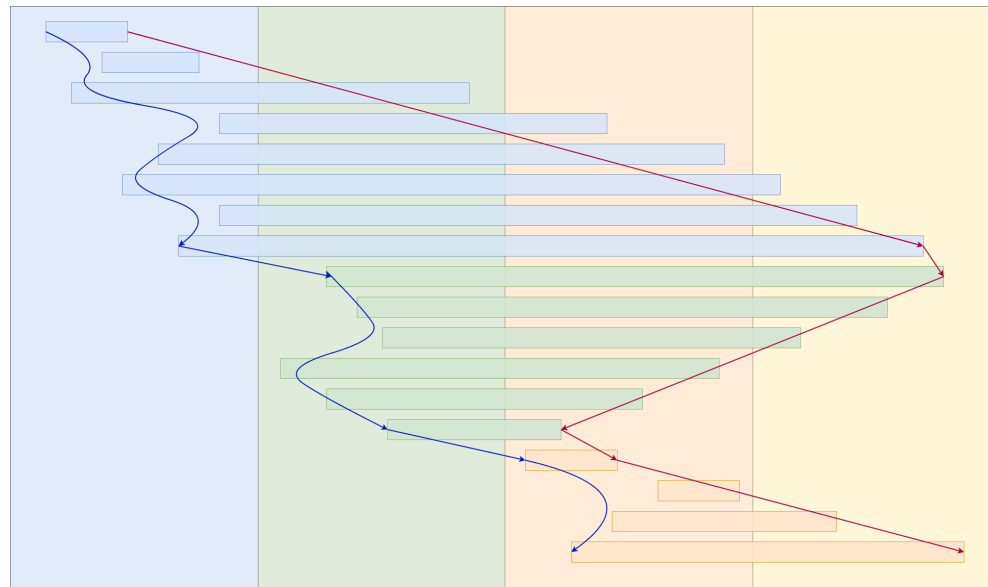


Figure 3: 奇偶块交错排序优化的莫队算法

3.2 时间复杂度

在 N 与 M 同阶时，莫队算法的时间复杂度为 $O(N\sqrt{N})$ 。其中，排序的时间复杂度为 $O(N \log N)$ ；区间之间转移的总时间复杂度为 $O(N\sqrt{N})$ 。

其中，对于区间之间转移的时间复杂度，我们可以通过以下方法推得：

令每一块中 l 的最大值为 $max_1, max_2, max_3, \dots, max_{\lceil \sqrt{N} \rceil}$ 。由第一次排序可知， $max_1 \leq max_2 \leq \dots \leq max_{\lceil \sqrt{N} \rceil}$ 。显然，对于每一块暴力求出第一个询问的时间复杂度为 $O(N)$ 。考虑最坏的情况，在每一块中， r 的最大值均为 N ，每次修改操作均要将 l 由 max_{i-1} 修改至 max_i 或由 max_i 修改至 max_{i-1} 。考虑 r ：因为 r 在块中已经排好序，所以在同一块修改完它的时间复杂度为 $O(N)$ 。对于所有块就是 $O(N\sqrt{N})$ 。重点分析 l ：因为每一次改变的时间复杂度都是 $O(max_i - max_{i-1})$ 的，所以在同一块中时间复杂度为 $O(\sqrt{N} \cdot (max_i - max_{i-1}))$ 。

3.2 时间复杂度

将每一块 l 的时间复杂度合在一起，通过裂项求和可得：

$$\begin{aligned} O(l) &= O\left(\sqrt{N}(max_1 - 1) + \sqrt{N}(max_2 - max_1) + \dots + \sqrt{N}(max_{\lceil \sqrt{N} \rceil} - max_{\lceil \sqrt{N} \rceil - 1})\right) \\ &= O\left(\sqrt{N} \cdot (max_1 - 1 + max_2 - max_1 + \dots + max_{\lceil \sqrt{N} \rceil} - max_{\lceil \sqrt{N} \rceil - 1})\right) \\ &= O\left(\sqrt{N} \cdot max_{\lceil \sqrt{N} \rceil - 1}\right) \end{aligned}$$

又因为 $max_{\lceil \sqrt{N} \rceil - 1}$ 最大为 N ，因此 l 的总时间复杂度最坏情况下为 $O(N\sqrt{N})$ 。

3.3 优化

3.3.1 最优块的大小

与分块有关的算法，块的大小对常数时间复杂度有很大的影响。设块长度为 S ，那么对于任意多个在同一块内的询问，指针的移动的距离为 N ，共有 $\frac{N}{S}$ 个块，移动的总距离就是 $\frac{N^2}{S}$ 。由于移动可能跨越块，所以还要加上 $O(MS)$ ，总复杂度为 $O\left(\frac{N^2}{S} + MS\right)$ 。当 S 取 $\frac{N}{\sqrt{M}}$ 时时间复杂度最优，为 $O(N\sqrt{M})$ 。

块的大小对莫队算法的时间复杂度影响非常显著。例如， M 与 $\sqrt{N}$ 同阶时，最优的块大小 $S = \frac{N}{\sqrt{M}} = N^{1-\frac{1}{2} \times \frac{1}{2}} = N^{\frac{3}{4}}$ ，时间复杂度可以达到 $O(N^{\frac{5}{4}})$ 。如果块的大小取 $\sqrt{N}$ ，其时间复杂度只有 $O(N\sqrt{N})$ 。

在本题中， N 与 M 同阶，因此块的大小取 $\sqrt{N}$ 即可。

3.3 优化

3.3.2 奇偶块交错排序

同一个块内的询问区间的右端点 r 均是递增的。在两个块之间转移时，指针 q 的移动距离较远。因此，我们可以做如下优化：对于编号为奇数的块，块内的询问区间按照 r 递增排序；对于编号为偶数的块，块内的询问区间按照 r 递减排序。这样，指针 q 在两个块之间的移动距离将会减小，可以在一定程度上优化常数时间复杂度，见 Figure 2 与 Figure 3。使用该优化，在洛谷的测试结果可以提高 8 分。

3.3 优化

3.3.3 数据离散化

由于该题输入数据 $a_i \in [1, 10^6]$ ，范围较大。因此，我们可以考虑将输入序列的值进行离散化。经过离散化后，对 cnt 数组的访问和修改更加连续，每次 p, q 指针的转移速度更快。由于莫队算法的瓶颈在于 p, q 指针的转移，因此经过离散化后莫队算法的常数时间复杂度得以有效减少。使用该优化，在洛谷的测试结果可以提高 48 分。

3.3 优化

3.3.4 优化 *cnt* 数组

通过上一种优化方法，我们可以发现，*cnt* 数组的访问和修改较为耗时。实际上，我们可以不记录 *cnt* 数组。考虑在读入时预处理 a_i 上一次出现的位置 bef_i 和下一次出现的位置 nxt_i 。在转移指针 p, q 时，只需要访问 bef_i 或 nxt_i 中的一个元素便可知道数 a_i 是否在原区间 $[p, q]$ 出现过，且无需进行对 bef 或 nxt 的修改操作。另外，在使用此方法优化时，离散化输入序列的值就没有必要了。使用该优化，在洛谷测试结果可以提高 60 分，顺利通过该题，最大的数据甚至可以在 1.01s 内通过。

3.4 衍生题目

本题的数据对莫队算法进行了一定的卡常，不适合作为莫队算法的入门练习。但近年来，不少省选题可以通过一定的优化，使用莫队算法解决。相比正解，莫队算法的思路和实现较为简便。因此，本题可作为莫队算法常数优化的一道良好的练习题。

以下提供几道普通莫队算法的模板题，可供读者快速掌握莫队算法，更好地理解莫队算法的核心思想：

- 洛谷 P2709 小 B 的询问
- 洛谷 P1494 [国家集训队] 小 Z 的袜子
- 洛谷 P4462 [CQOI2018] 异或序列

Outline

1. 例题
2. 分块
3. 普通莫队算法
4. 树状数组 / 线段树
5. 归并树
6. 扫描线
7. 总结

4.1 实现

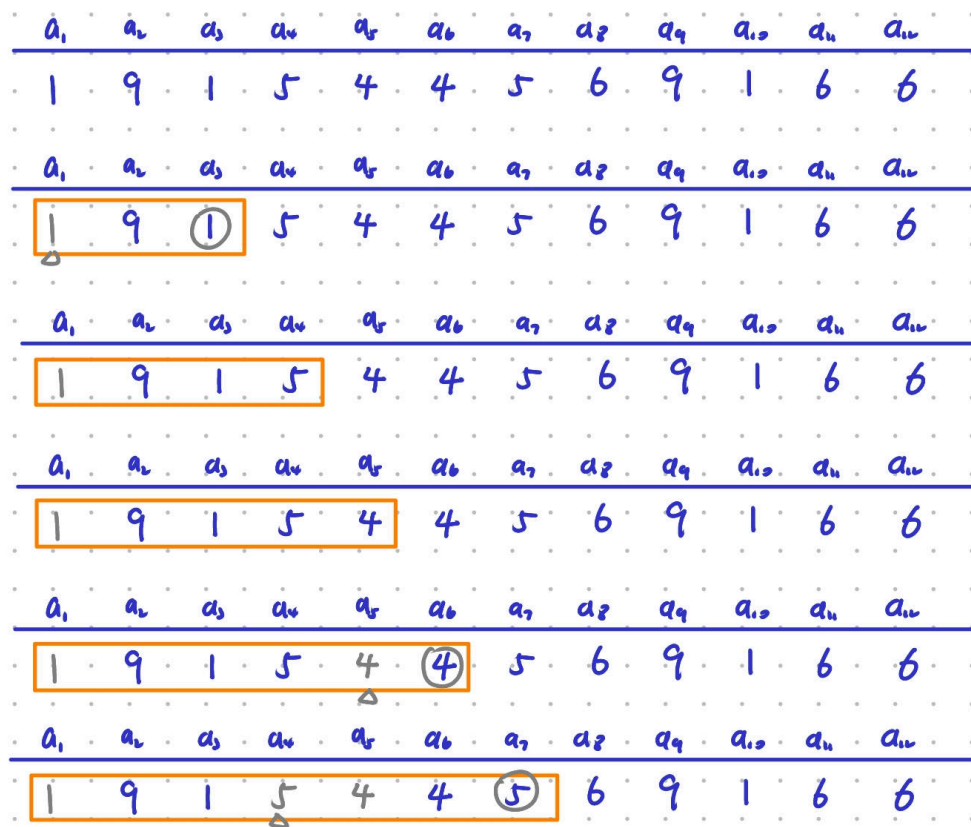


Figure 4: 树状数组 / 线段树解法简要示意图

4.2 时间复杂度

该算法的时间复杂度为 $O((N + M) \log N) = O(N \log N)$ ，其中 N 为序列长度， M 为询问区间的数量。预处理 $\{bef_i\}$ 的时间复杂度为 $O(N)$ 。将查询按照右端点 r 排序的时间复杂度为 $O(M \log M)$ 。对于所有查询，更新树状数组或线段树的时间复杂度为 $O((N + M) \log N)$ 。

使用树状数组 / 线段树实现本题游刃有余，不需要过多的优化，其关键在于如何对询问离线，以及树状数组 / 线段树维护什么内容。与莫队算法类似，其核心在于对询问进行离线处理。

Outline

1. 例题
2. 分块
3. 普通莫队算法
4. 树状数组 / 线段树
5. 归并树
6. 扫描线
7. 总结

5.1 实现

在读入序列 $\{a_i\}$ 时预处理每个元素 a_i 在序列中上一次出现的位置 bef_i 。

利用归并树来组织 bef 数组的数据。归并树是一棵线段树，每个节点负责一段区间，存储该区间内 bef 值的有序序列。具体实现时，从根节点开始递归划分区间。对于当前节点负责的区间 $[l, r]$ ，如果 $l = r$ ，说明是叶子节点，直接将 $bef[l]$ 存入该节点的数组中。如果 $l < r$ ，则将区间分为 $[l, mid]$ 和 $[mid + 1, r]$ 两部分，分别递归构建左子树和右子树。子树构建完成后，将左右子树的有序 bef 序列归并到当前节点的序列中。归并的过程类似于归并排序，使用双指针依次比较并合并，确保当前节点存储的 bef 序列也是有序的。由于每个元素只会被归并 $O(\log N)$ 次，建树总时间复杂度为 $O(N \log N)$ 。

对于每次查询，给定区间 $[l, r]$ ，需要在归并树上计算答案。查询函数从根节点开始，判断当前节点负责的区间 $[x, y]$ 与查询区间 $[l, r]$ 的关系。如果 $[x, y]$ 完全被 $[l, r]$ 包含，则直接在该节点的有序 bef 序列中使用二分查找，统计小于 l 的 bef 值的个数，这些位置对应的元素在 $[l, r]$ 内是首次出现。如果 $[x, y]$ 与 $[l, r]$ 部分重叠，则递归查询左子树（如果 $l \leq mid$ ）和右子树（如果 $mid < r$ ），将两部分的结果相加。

5.2 时间复杂度

归并树算法的时间复杂度为 $O((N + M) \log^2 N) = O(N \log^2 N)$ ，其中 N 为序列长度， M 为询问区间的数量。构建归并树的时间复杂度为 $O(N \log N)$ ；对于每个询问，我们需要在 $O(\log N)$ 个节点上进行二分查找，每个节点内二分的时间复杂度为 $O(\log N)$ ，因此对于单个询问的时间复杂度为 $O(M \log^2 N)$ 。

5.3 优化

朴素的实现仍需在 $O(\log N)$ 个节点上进行二分查找，总复杂度为 $O(M \log^2 N)$ ，可能面临常数过大的风险，在洛谷的测试结果为 88 分。考虑应用离线处理和优化技巧：将所有询问按左端点 l 递增排序，并在每个节点记录上一次二分查找的结果 d 。由于 l 单调递增，下次查询时可从 $d + 1$ 开始二分，缩小查找范围，从而降低平均时间复杂度。通过这样的优化，可以有效提高性能。

Outline

1. 例题
2. 分块
3. 普通莫队算法
4. 树状数组 / 线段树
5. 归并树
- 6. 扫描线**
7. 总结

6.1 实现

i	a_i	bef_i
1	1	0
2	9	0
3	1	1
4	5	0
5	4	0
6	4	5
7	5	4
8	6	0
9	9	2
10	1	3
11	6	8
12	6	11
13	6	12
14	9	9
15	5	7

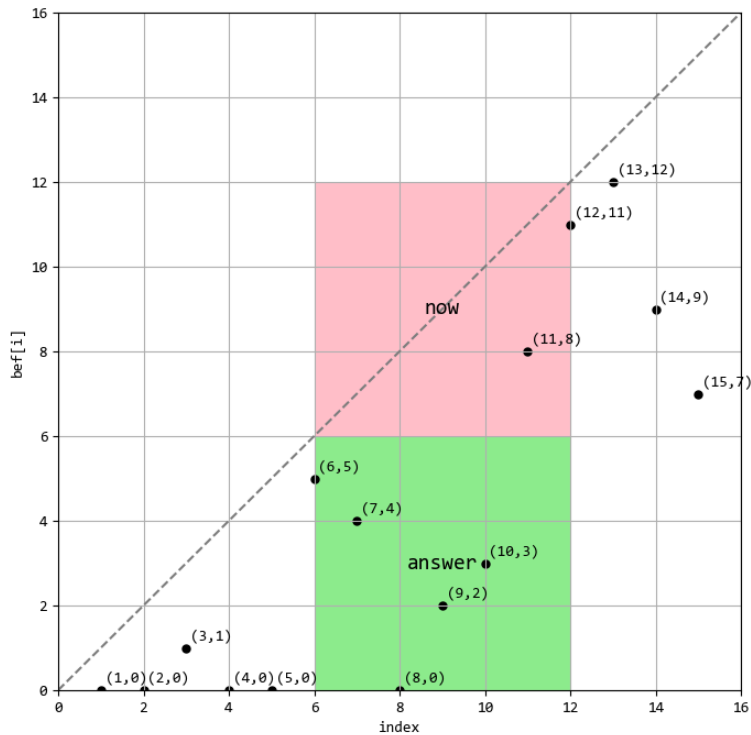


Figure 5: 转化为二维数点问题

6.2 时间复杂度

该算法的时间复杂度为 $O((N + M) \log(N + M)) = O(N \log N)$ 。预处理 bef 数组时间复杂度 $O(N)$ ；将询问操作和加点操作排序时间复杂度 $O((2M + N) \log(2M + N))$ ，每次树状数组 / 线段树的修改和查询操作的时间复杂度均为 $O(\log N)$ ，共有 N 次加点操作和 $2M$ 次查询操作。

Outline

1. 例题
2. 分块
3. 普通莫队算法
4. 树状数组 / 线段树
5. 归并树
6. 扫描线
7. 总结

7.1 总结

本文围绕区间不同元素个数查询问题，系统分析了分块、普通莫队算法、树状数组/线段树、归并树和扫描线五种经典算法。通过对算法复杂度、实现难度、优化方案的深入分析，我们可以发现每种算法各有特点，适用于不同场景。从时间复杂度角度看，树状数组/线段树和扫描线的复杂度为 $O(N \log N)$ ，理论上效率最高；分块和莫队算法次之，为 $O(N\sqrt{N})$ ；归并树虽然复杂度达到 $O(N \log^2 N)$ ，但在某些特定场景下表现同样优异。空间复杂度方面，分块、莫队算法为 $O(N)$ ，而归并树和扫描线需要 $O(N \log N)$ 的额外空间。这些差异使得在不同的应用场景下，选择合适的算法至关重要。

7.1 总结

在信息学奥赛中，这些算法是解决区间查询问题的基础工具。近年来，省选和 NOI 试题中频繁出现需要高效处理区间统计信息的问题。选手需根据题目特点和个人熟练度选择合适算法，权衡时间复杂度和实现难度。特别值得一提的是，这些算法思想可以迁移到树上问题和图论问题中，如树上莫队、树上分块等高级技巧，进一步扩展了应用范围。除基础的区间查询外，这些算法还可以处理带修改的区间查询、二维区间查询等更复杂的变形，通过组合多种数据结构，如莫队结合树状数组、分块结合线段树等方式，解决更复杂的问题。对于竞赛选手来说，掌握这些算法不仅是解题的工具，更是培养算法思维的重要途径。

7.1 总结

通过本文的系统分析，我们不仅学习了解决具体问题的多种方法，更重要的是理解了不同算法设计思想的本质和适用条件。在算法选择时，需综合考虑数据规模、查询特点、时间要求、空间限制等多种因素。例如，对于小规模数据或对实时性要求较高的场景，可能倾向于选择实现简单的树状数组；对于大规模数据的离线处理，莫队算法或扫描线可能是更好的选择；而面对复杂多变的查询需求，分块算法的灵活性则显得尤为重要。无论在竞赛还是工程中，掌握多种算法工具，并能根据具体问题特点进行有针对性的选择和优化，是解决大规模数据处理问题的关键。通过不断学习和实践，我们能够更加熟练地运用这些算法工具，解决更为复杂的实际问题。

7.2 参考文献

- [1] KONNYAKUXZY. OI Wiki - 莫队算法 [EB/OL]. (2018-07-30). <https://oi-wiki.org/misc/mo-algo/> .
- [2] OOJ_BAI. 重新编号颜色优化莫队算法 [EB/OL]. (2024-09-19). <https://www.luogu.com.cn/discuss/929927> .
- [3] PHANTOM_LANDLER. 通过优化桶优化莫队算法 [EB/OL]. (2024-11-25). <https://www.luogu.com.cn/discuss/1005310> .

7.3 致谢

- 感谢「沪疆教育信息化 算法小论文」提供的学习和交流的平台。
- 感谢张卫国老师、诸峰老师对我的指导和帮助。
- 特别感谢程佳润同学对我的支持。
- 感谢洛谷平台提供的题目和测试。